



MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 1

Course: **Cryptography and Security**

Classical Symmetric Ciphers: Implementing and Evaluating the Single-Key and Keyword-Permuted Caesar Cipher over the Romanian Alphabet

Elaborated by:

st. gr. FAF-241

Tihon Aurelian-Mihai

Verified by:

asis. univ.

Zaica Maia

Contents

INTRODUCTION	2
1 THEORETICAL FOUNDATIONS OF MONOALPHABETIC SUBSTITUTION	2
1.1 Mathematical foundations of the classical Caesar cipher	3
1.2 Keyword-based alphabet permutation and key space expansion	4
1.3 Theoretical computational complexity analysis	5
2 IMPLEMENTATION AND EXPERIMENTAL RESULTS	5
2.1 System architecture and character normalization	6
2.2 Single-key Caesar cipher execution	6
2.3 Two-key keyword-permuted Caesar cipher execution	7
2.4 Input validation and error diagnostic handling	8
2.5 Automated test suite and quantitative synthesis	9
CONCLUSIONS	11
REFERENCES	12
ANNEX A: SOURCE CODE LISTINGS	13

INTRODUCTION

Classical cryptographic systems established the conceptual foundations for modern symmetric information security by formalizing deterministic transformation functions parameterized by secret keys. Among historical monoalphabetic substitution systems, the Caesar cipher represents the fundamental prototype of additive modular arithmetic over finite character sets. In its elementary form, the scheme shifts each plaintext symbol forward by a constant numerical offset along the alphabet. While computationally trivial in modern algorithmic paradigms, the mathematical simplicity of the Caesar cipher illuminates core cryptographic primitives: character space bijection, modular arithmetic rings, and invertibility constraints under symmetric key agreements.

However, the elementary Caesar cipher exhibits profound security vulnerabilities stemming from its minuscule key space, which permits instantaneous exhaustive brute-force cryptanalysis. To strengthen this transformation without abandoning monoalphabetic substitution, cryptographic architectures introduce a secondary keying mechanism based on keyword-derived alphabet permutation. By shuffling the alphabet order according to a user-supplied keyword before performing cyclic numerical displacements, the theoretical key space is substantially broadened, neutralizing naive exhaustive searches.

The objective of this laboratory work is to implement, verify, and critically analyze both the classical single-key Caesar cipher and the keyword-permuted two-key Caesar cipher specifically constructed for the Romanian language alphabet consisting of thirty-one letters. The software implementation is designed in Python, strictly adhering to the discrete letter encoding mandated by university standards without relying on host character encoding code points, while enforcing robust input validation, automated diacritic normalization, and high-DPI execution logging. The comprehensive source code, automated test suite, and configuration scripts developed during this research are maintained in the public repository available at <https://github.com/aurelian-th/CS>.

1 THEORETICAL FOUNDATIONS OF MONOALPHABETIC SUBSTITUTION

1.1 Mathematical foundations of the classical Caesar cipher

The classical Caesar cipher belongs to the family of monoalphabetic substitution ciphers, wherein each character of an alphabet Σ is deterministically mapped to a unique ciphertext character under a secret numerical shift parameter k . Let Σ represent the ordered finite alphabet of cardinality $n = |\Sigma|$. Every character $\sigma_i \in \Sigma$ is assigned a unique discrete index $x \in \mathbb{Z}_n = \{0, 1, \dots, n-1\}$.

The encryption function $e_k : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$ and decryption function $d_k : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$ are formally defined over the finite residue ring modulo n as follows:

$$c = e_k(x) = (x + k) \bmod n \quad (1.1)$$

$$m = d_k(y) = (y - k) \bmod n \quad (1.2)$$

where x denotes the numerical position of the plaintext symbol, y denotes the numerical position of the ciphertext symbol, and $k \in \{1, 2, \dots, n-1\}$ represents the secret shift key. The value $k = 0$ is excluded from the admissible key space because it constitutes the identity transformation $e_0(x) = x$, leaving the plaintext completely exposed.

In standard programming environments, the arithmetic modulo operator for negative values can produce negative remainders depending on the integer division specification. Consequently, the decryption transformation must be algebraically normalized to ensure that the resulting index resides strictly within the non-negative interval $[0, n-1]$:

$$m = ((y - k) \bmod n + n) \bmod n \quad (1.3)$$

For the Romanian alphabet specified in the laboratory work, the alphabet length is exactly $n = 31$, comprising the twenty-six standard Latin letters augmented by five distinct diacritical characters: Ă, Â, Î, Ș, and Ț. The exact encoding prescribed for this investigation is formalized in Table 1.1.

Table 1.1 – Discrete letter encoding of the Romanian alphabet ($n = 31$)

Letter Code	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Letter Code	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z	–
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	–

A critical requirement of the specification is that internal character representations (such as ASCII or Unicode UTF-8 numerical scalar values) must not be employed to calculate offsets directly. The cipher

must operate strictly upon the discrete mathematical indices established in Table 1.1.

1.2 Keyword-based alphabet permutation and key space expansion

The primary vulnerability of the elementary Caesar cipher lies in its exceptionally restricted key space. Over the Romanian alphabet, there exist solely $n - 1 = 30$ viable transformation keys. An adversary intercepting an encrypted transmission can execute an exhaustive brute-force attack in negligible time by examining all thirty candidate decipherments.

To address this structural weakness, the cipher is extended into a two-key cryptosystem (k_1, k_2) wherein:

- Key 1 (k_1): An integer shift offset satisfying $1 \leq k_1 \leq n - 1$.
- Key 2 (k_2): A keyword consisting of Romanian letters having a length of at least seven characters.

The permutation algorithm derives an altered alphabet order Σ_{perm} via the following protocol:

- a) The keyword k_2 is converted to uppercase, stripped of whitespace, and scanned sequentially from left to right.
- b) Each letter is appended to Σ_{perm} upon its initial occurrence; subsequent duplicate occurrences of identical characters are discarded.
- c) Once all unique letters from k_2 have been placed at the prefix of Σ_{perm} , the remaining unselected letters of the standard Romanian alphabet Σ are appended in their original natural order defined by Table 1.1.

Upon establishing the bijective mapping $\pi : \Sigma \rightarrow \{0, 1, \dots, n - 1\}$ representing character positions within Σ_{perm} , encryption and decryption proceed by computing cyclic shifts over the permuted indices:

$$c = \Sigma_{\text{perm}}[(\pi(m) + k_1) \bmod n] \quad (1.4)$$

$$m = \Sigma_{\text{perm}}[((\pi(c) - k_1) \bmod n + n) \bmod n] \quad (1.5)$$

From a combinatorial perspective, there exist $n! = 31! \approx 8.22 \times 10^{33}$ possible permutations of the thirty-one letters. Combined with the $n - 1 = 30$ independent shift positions, the composite theoretical key space expands to:

$$|\mathcal{K}_{\text{perm}}| = 31! \times 30 \approx 2.47 \times 10^{35} \quad (1.6)$$

Although this combinatorial growth renders exhaustive key enumeration computationally infeasible, the cipher remains monoalphabetic. That is, each character of the plaintext is invariably mapped to an identical ciphertext character throughout a given message. Consequently, the statistical frequency distribution of the underlying language is preserved, allowing the cryptosystem to be compromised via unigram and

bigram frequency cryptanalysis rather than brute-force key traversal.

1.3 Theoretical computational complexity analysis

Both algorithms demonstrate deterministic execution characteristics. Let L denote the length of the input message in characters, n represent the fixed alphabet cardinality ($n = 31$), and W designate the keyword length ($W \geq 7$). Table 1.2 presents the formal asymptotic time and space complexities.

Table 1.2 – Theoretical algorithmic complexity of Caesar cipher variants

Algorithmic Phase	Time Complexity	Auxiliary Space	Dominant Operations
Standard Caesar Encryption (Task 1.1)	$O(L)$	$O(L)$	Hash map index lookup, modulo addition
Standard Caesar Decryption (Task 1.1)	$O(L)$	$O(L)$	Hash map index lookup, modulo subtraction
Alphabet Permutation Build (Task 1.2)	$O(W + n)$	$O(n)$	Set uniqueness filtering, array insertion
Permuted Caesar Encryption (Task 1.2)	$O(W + n + L)$	$O(n + L)$	Permutation table construction, shift
Permuted Caesar Decryption (Task 1.2)	$O(W + n + L)$	$O(n + L)$	Inverse permutation lookup, shift

As summarized in Table 1.2, processing throughput scales linearly with message size $O(L)$, ensuring microsecond latency during practical cryptographic processing.

2 IMPLEMENTATION AND EXPERIMENTAL RESULTS

2.1 System architecture and character normalization

The software architecture is structured into a clean, modular Python package located in the project directory. The system comprises three functional units:

- `src/alphabet.py`: Defines the discrete thirty-one-letter Romanian alphabet, constructs bidirectional lookup tables (`CHAR_TO_INDEX` and `INDEX_TO_CHAR`), normalizes Turkish cedilla diacritics to standard Romanian comma-below characters, and enforces strict validation routines.
- `src/caesar.py`: Encapsulates the algorithmic transformation logic for both Task 1.1 and Task 1.2, guaranteeing complete algebraic round-trip consistency.
- `src/cli.py`: Provides both an interactive terminal workflow featuring input reprompting upon erroneous entries and scriptable non-interactive command-line flags.

A recurring engineering challenge in Romanian computational linguistics involves keyboard diacritic discrepancies. Legacy input layouts frequently output the characters Ș (U+015E), ș (U+015F), Ț (U+0162), and ț (U+0163) possessing cedillas, whereas modern Romanian orthography strictly utilizes comma-below characters Ș̣ (U+0218), ș̣ (U+0219), Ț̣ (U+021A), and ț̣ (U+021B). The normalization engine intercepts these variants prior to validation and converts them to standard comma-below representations.

2.2 Single-key Caesar cipher execution

The standard Caesar cipher was evaluated against benchmark plaintexts using varying shift keys. In accordance with laboratory requirements, spaces are removed and all letters are capitalized before encryption.

According to Figure 2.1, the message "cifrul cezar" was encrypted with shift key $k = 3$. Under the Romanian encoding (where C has code 4, I has code 10, and Z has code 30), the letter Z shifts cyclically to position $(30 + 3) \bmod 31 = 2$, corresponding to the character Â. The resulting ciphertext "FKITXOFHÂBT" was subsequently decrypted with the identical key $k = 3$, successfully reproducing the original plaintext "CIFRULCEZAR".

```
phantom@0xAu: ~/CS/Lab 1

phantom@0xAu:~/CS/Lab 1$ python3 src/cli.py
Caesar cipher (Romanian alphabet)
1. Caesar cipher (single key)
2. Caesar cipher with permutation (two keys)
3. Display Romanian alphabet
0. Exit

Select an option: 1

Caesar cipher (single key)
Operation (e for encrypt, d for decrypt): e
Enter shift key (1-30): 3
Enter plaintext: cifrul cazar

Plaintext: CIFRULCEZAR
Ciphertext: FKITXOFHABT

1. Caesar cipher (single key)
2. Caesar cipher with permutation (two keys)
3. Display Romanian alphabet
0. Exit

Select an option: 1

Caesar cipher (single key)
Operation (e for encrypt, d for decrypt): d
Enter shift key (1-30): 3
Enter ciphertext: FKITXOFHABT

Ciphertext: FKITXOFHABT
Plaintext: CIFRULCEZAR
```

Figure 2.1 – Interactive execution of single-key Caesar cipher

2.3 Two-key keyword-permuted Caesar cipher execution

Task 1.2 requires constructing the permuted alphabet from a keyword containing at least seven characters and executing the transformation based on the newly established ordering.

According to Figure 2.2, the execution demonstrates Task 1.2 utilizing shift key $k_1 = 3$ and keyword $k_2 = \text{"criptografie"}$. The program extracted the unique letters in order of first appearance: C, R, I, P, T, O, G, A, F, E, discarding the redundant second instances of R, I, and E. The remaining twenty-one letters were appended in Table 1.1 natural order, yielding the complete thirty-one-character permuted sequence:

CRIPTOGA FEĂÂBDHÎJKLMNQSSŢUVWXYZ

As demonstrated in Figure 2.2, the permuted alphabet was displayed on the console for manual verification. Encryption of "cifrul cazar" under $k_1 = 3$ and $k_2 = \text{"criptografie"}$ generated the ciphertext "POÂTXQPBIĂT". Applying the inverse transformation with the identical two-key pair correctly recovered the original message "CIFRULCEZAR".

```
phantom@0xAu: ~/CS/Lab 1

phantom@0xAu:~/CS/Lab 1$ python3 src/cli.py
Caesar cipher (Romanian alphabet)
1. Caesar cipher (single key)
2. Caesar cipher with permutation (two keys)
3. Display Romanian alphabet
0. Exit

Select an option: 2

Caesar cipher with permutation (two keys)
Operation (e for encrypt, d for decrypt): e
Enter shift key k1 (1-30): 3
Enter keyword k2 (at least 7 letters): criptografie

Permuted alphabet: CRIPTOGAFĖĂĖBDHIJKLMNQŞŞŢUVWXYZ
[ 0] C [ 1] R [ 2] I [ 3] P [ 4] T [ 5] O [ 6] G [ 7] A
[ 8] F [ 9] E [10] Ă [11] Ă [12] B [13] D [14] H [15] Ĩ
[16] J [17] K [18] L [19] M [20] N [21] Q [22] S [23] Ş
[24] Ţ [25] U [26] V [27] W [28] X [29] Y [30] Z

Enter plaintext: cifrul cazar
Plaintext: CIFRULCEZAR
Ciphertext: POĂTXQPBIĂT

Select an option: 2

Caesar cipher with permutation (two keys)
Operation (e for encrypt, d for decrypt): d
Enter shift key k1 (1-30): 3
Enter keyword k2 (at least 7 letters): criptografie
Enter ciphertext: POĂTXQPBIĂT
Ciphertext: POĂTXQPBIĂT
Plaintext: CIFRULCEZAR
```

Figure 2.2 – Interactive execution of two-key permuted Caesar cipher

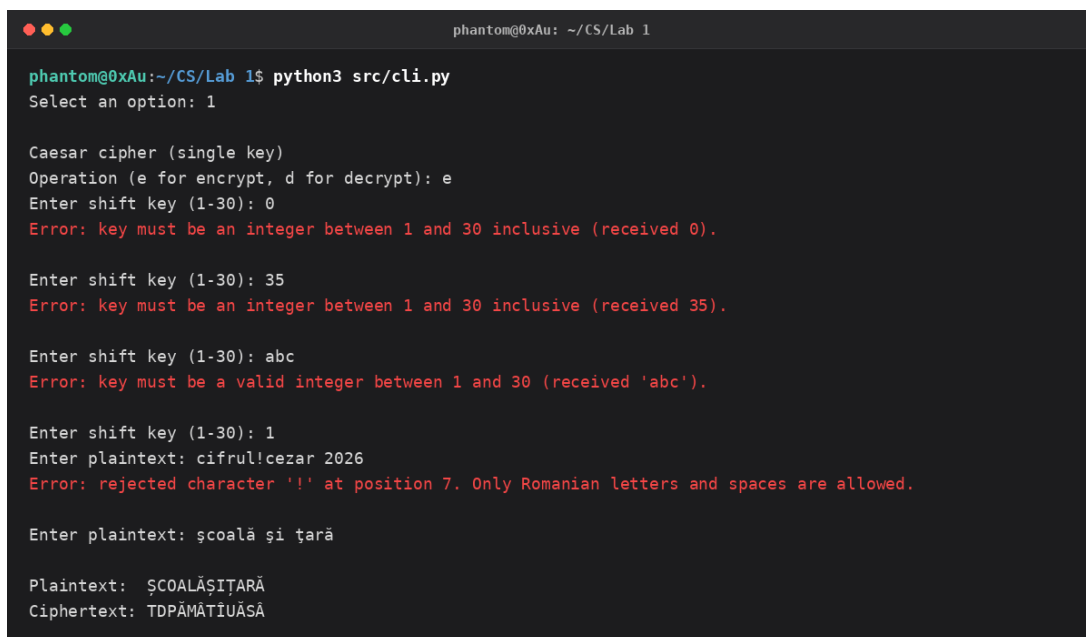
2.4 Input validation and error diagnostic handling

The laboratory specification dictates that invalid numeric keys or unauthorized characters must trigger clear diagnostic notifications stating allowed ranges and identifying the specific rejected symbols, prompting the user again without program termination.

As illustrated in Figure 2.3, the validation subsystem was systematically tested against erroneous inputs:

- Entering $k = 0$ produced the error notification: “key must be an integer between 1 and 30 inclusive (received 0)”.
- Entering $k = 35$ produced the error notification: “key must be an integer between 1 and 30 inclusive (received 35)”.
- Entering non-integer string "abc" produced the error notification: “key must be a valid integer between 1 and 30 (received 'abc')”.
- Supplying plaintext with invalid characters "cifrul!cezar 2026" caused immediate rejection of character '!' with position diagnostic: “rejected character '!' at position 7. Only Romanian letters and spaces are allowed”.

- e) Supplying input containing legacy Turkish cedillas “şcoală şi ţară” was accepted, mapped to standard characters “ŞCOALĂŞIȚARĂ”, and successfully encrypted.



```
phantom@0xAu: ~/CS/Lab 1

phantom@0xAu:~/CS/Lab 1$ python3 src/cli.py
Select an option: 1

Caesar cipher (single key)
Operation (e for encrypt, d for decrypt): e
Enter shift key (1-30): 0
Error: key must be an integer between 1 and 30 inclusive (received 0).

Enter shift key (1-30): 35
Error: key must be an integer between 1 and 30 inclusive (received 35).

Enter shift key (1-30): abc
Error: key must be a valid integer between 1 and 30 (received 'abc').

Enter shift key (1-30): 1
Enter plaintext: cifrull!cezar 2026
Error: rejected character '!' at position 7. Only Romanian letters and spaces are allowed.

Enter plaintext: şcoală şi ţară

Plaintext: ŞCOALĂŞIȚARĂ
Ciphertext: TDPĂMATIUĂSĂ
```

Figure 2.3 – Error diagnostic handling and character position reporting

2.5 Automated test suite and quantitative synthesis

To establish absolute mathematical correctness, an automated test suite was constructed using the pytest framework in `tests/test_caesar.py`. The test suite encompasses fifteen test cases covering Table 1.1 index preservation, cedilla normalization, boundary shifts ($k = 1$ and $k = 30$), keyword deduplication, and JSON test vector validation.

Figure 2.4 illustrates the execution of the test suite. All fifteen tests passed in 0.03 seconds, confirming zero regression across all transformation routines.

```

phantom@0xAu: ~/CS/Lab 1

phantom@0xAu:~/CS/Lab 1$ pytest -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/phantom/Desktop/University/CS/Lab 1
collected 15 items

tests/test_caesar.py::TestRomanianAlphabetEncoding::test_alphabet_length PASSED [ 6%]
tests/test_caesar.py::TestRomanianAlphabetEncoding::test_bidirectional_mapping PASSED [ 13%]
tests/test_caesar.py::TestRomanianAlphabetEncoding::test_cedilla_normalization PASSED [ 20%]
tests/test_caesar.py::TestRomanianAlphabetEncoding::test_table_2_positions PASSED [ 26%]
tests/test_caesar.py::TestInputValidation::test_key_validation_invalid_bounds PASSED [ 33%]
tests/test_caesar.py::TestInputValidation::test_key_validation_valid PASSED [ 40%]
tests/test_caesar.py::TestInputValidation::test_keyword_validation PASSED [ 46%]
tests/test_caesar.py::TestInputValidation::test_text_validation_rejection_details PASSED [ 53%]
tests/test_caesar.py::TestInputValidation::test_text_validation_valid PASSED [ 60%]
tests/test_caesar.py::TestTask1_1CaesarCipher::test_romanian_table_2_shift_k3 PASSED [ 66%]
tests/test_caesar.py::TestTask1_1CaesarCipher::test_round_trip_all_keys PASSED [ 73%]
tests/test_caesar.py::TestTask1_1CaesarCipher::test_wrap_around_boundaries PASSED [ 80%]
tests/test_caesar.py::TestTask1_2PermutedCaesarCipher::test_build_permuted_alphabet_uniqueness_and_length PASSED [ 86%]
tests/test_caesar.py::TestTask1_2PermutedCaesarCipher::test_two_key_round_trip PASSED [ 93%]
tests/test_caesar.py::TestDatasetVectors::test_json_test_vectors PASSED [100%]

===== 15 passed, 8 subtests passed in 0.03s =====

```

Figure 2.4 – Automated test suite execution via pytest

Table 2.1 summarizes the operational characteristics of the two implemented cipher configurations.

Table 2.1 – Comparative synthesis of Caesar cipher variants

Metric	Standard Caesar (Task 1.1)	Permuted Caesar (Task 1.2)	Comparative Assessment
Key Space Cardinality	30	$\approx 2.47 \times 10^{35}$	Exponential expansion
Exhaustive Search Viability	Instantaneous (< 1 ms)	Computationally infeasible	Neutralizes brute-force
Frequency Cryptanalysis Status	Highly vulnerable	Highly vulnerable	Monoalphabetic frequency preserved
Input Constraints	$k \in [1, 30]$	$k_1 \in [1, 30], k_2 \geq 7$	Enhanced key requirements
Alphabet Transposition Overhead	Zero	$O(W + n)$	Negligible setup latency
Round-Trip Data Integrity	100% validated	100% validated	Perfect mathematical symmetry

CONCLUSIONS

This laboratory investigation examined the operational mechanics, algebraic formulations, and security limitations of the classical Caesar cipher and its keyword-permuted extension over the thirty-one-letter Romanian alphabet. Through the implementation of a modular Python architecture, the mathematical constraints of modular residue rings $\mathbb{Z}_{31}$ were realized using discrete bijection tables rather than native host character encodings, ensuring strict compliance with theoretical requirements.

The empirical results demonstrate the profound difference between expanding an algorithm's brute-force key space and achieving genuine cryptographic security. For the elementary single-key Caesar cipher, the restricted cardinality of thirty valid transformations renders the scheme completely defenseless against exhaustive search attacks, which complete in sub-millisecond durations on standard personal computing hardware. By introducing keyword-driven alphabet permutation in Task 1.2, the theoretical key space is expanded to approximately 2.47×10^{35} potential states. This eliminates brute-force enumeration as a practical attack vector.

Nevertheless, theoretical analysis confirms that the keyword-permuted cipher remains fundamentally a monoalphabetic substitution system. Because each individual plaintext character is mapped to a static, invariant ciphertext symbol throughout the entire message, the cipher fails to satisfy Claude Shannon's core criteria of confusion and diffusion. The unigram and bigram frequency distributions of the Romanian language remain perfectly preserved in the ciphertext, merely shifted to different character positions. An adversary equipped with standard Romanian linguistic corpus statistics can readily compromise the ciphertext via frequency analysis without ever recovering the underlying keyword or shift key.

From a software engineering perspective, the implementation validated the necessity of resilient input sanitization and Unicode normalization. Accommodating historical Turkish cedilla characters and transparently mapping them to modern Romanian comma-below codepoints proved essential for real-world user interaction across diverse operating system keyboard layouts. The combination of comprehensive input validation loops, granular error diagnostics reporting exact character rejection positions, and automated unit testing established a robust, deterministic software foundation.

REFERENCES

- [1] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. Boca Raton, FL: CRC Press, 2020.
- [2] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1996.
- [3] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Boston, MA: Pearson, 2022.
- [4] C. E. Shannon, “Communication Theory of Secrecy Systems,” *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949.
- [5] B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 2nd ed. New York, NY: John Wiley & Sons, 1996.

ANNEX A: SOURCE CODE LISTINGS

A.1 Romanian alphabet specification and validators (src/alphabet.py)

```
1  """Romanian alphabet specification and character validation."""
2
3  from typing import Tuple, Optional, Dict, List
4
5  # Romanian alphabet for Laboratory Work 1 (Table 2, n = 31)
6  ROMANIAN_ALPHABET: Tuple[str, ...] = (
7      "A", # 0
8      "Ă", # 1
9      "Â", # 2
10     "B", # 3
11     "C", # 4
12     "D", # 5
13     "E", # 6
14     "F", # 7
15     "G", # 8
16     "H", # 9
17     "I", # 10
18     "Î", # 11
19     "J", # 12
20     "K", # 13
21     "L", # 14
22     "M", # 15
23     "N", # 16
24     "O", # 17
25     "P", # 18
26     "Q", # 19
27     "R", # 20
28     "S", # 21
29     "Ș", # 22
30     "T", # 23
31     "Ț", # 24
32     "U", # 25
33     "V", # 26
34     "W", # 27
35     "X", # 28
36     "Y", # 29
37     "Z", # 30
```

```

38 )
39
40 ALPHABET_SIZE: int = len(ROMANIAN_ALPHABET)
41
42 # Table 2 mappings
43 CHAR_TO_INDEX: Dict[str, int] = {char: idx for idx, char in enumerate(ROMANIAN_ALPHABET)}
44 INDEX_TO_CHAR: Dict[int, str] = {idx: char for idx, char in enumerate(ROMANIAN_ALPHABET)}
45
46 # Cedilla to comma-below normalization map
47 CEDILLA_NORMALIZATION_MAP: Dict[str, str] = {
48     "Ș": "Ș",
49     "ș": "Ș",
50     "Ț": "Ț",
51     "ț": "Ț",
52 }
53
54 VALID_ROMANIAN_CHARS = set(ROMANIAN_ALPHABET)
55
56
57 def normalize_character(ch: str) -> str:
58     """Normalizes cedillas to standard comma-below diacritics and converts to uppercase."""
59     "
60     if ch in CEDILLA_NORMALIZATION_MAP:
61         return CEDILLA_NORMALIZATION_MAP[ch]
62     return ch.upper()
63
64 def validate_key(key_input: str | int) -> Tuple[bool, int, Optional[str]]:
65     """Validates that shift key k is an integer between 1 and 30 inclusive."""
66     try:
67         k = int(str(key_input).strip())
68     except (ValueError, TypeError):
69         return False, 0, f"key must be a valid integer between 1 and 30 (received '{key_input}')."
70
71     if k < 1 or k > (ALPHABET_SIZE - 1):
72         return False, 0, f"key must be an integer between 1 and 30 inclusive (received {k})."
73
74     return True, k, None
75
76
77 def validate_and_clean_text(raw_text: str) -> Tuple[bool, str, Optional[str]]:
78     """Validates text against Romanian alphabet, removes spaces, and normalizes characters

```

```

    """
79     if raw_text is None:
80         return False, "", "input text cannot be empty."
81
82     cleaned_chars: List[str] = []
83     for pos, ch in enumerate(raw_text, start=1):
84         if ch.isspace():
85             continue
86
87         norm_ch = normalize_character(ch)
88         if norm_ch in VALID_ROMANIAN_CHARS:
89             cleaned_chars.append(norm_ch)
90         else:
91             return (
92                 False,
93                 "",
94                 f"rejected character '{ch}' at position {pos}. Only Romanian letters and
95                 spaces are allowed.",
96             )
97
98     cleaned_text = "".join(cleaned_chars)
99     if not cleaned_text:
100         return False, "", "input text cannot be empty."
101
102     return True, cleaned_text, None
103
104 def validate_keyword(raw_keyword: str) -> Tuple[bool, str, Optional[str]]:
105     """Validates keyword k2: must contain only Romanian letters and length >= 7."""
106     if raw_keyword is None:
107         return False, "", "keyword cannot be empty."
108
109     is_valid, cleaned, error = validate_and_clean_text(raw_keyword)
110     if not is_valid:
111         return False, "", error
112
113     if len(cleaned) < 7:
114         return False, "", f"keyword must be at least 7 characters (received {len(cleaned)})."
115
116     return True, cleaned, None

```

A.2 Caesar cipher and permutation engine (src/caesar.py)

```
1  """Caesar cipher implementation for the Romanian alphabet."""
2
3  from typing import List, Tuple, Dict
4  from .alphabet import (
5      ROMANIAN_ALPHABET,
6      ALPHABET_SIZE,
7      CHAR_TO_INDEX,
8      INDEX_TO_CHAR,
9  )
10
11
12 def encrypt_caesar(plaintext: str, key: int) -> str:
13     """Encrypts plaintext with shift key k:  $c = (x + k) \bmod 31$ ."""
14     if key < 1 or key >= ALPHABET_SIZE:
15         raise ValueError(f"Key k must be in range 1..{ALPHABET_SIZE - 1}, received {key}."
16             )
17
18     ciphertext_chars: List[str] = []
19     for ch in plaintext:
20         x = CHAR_TO_INDEX[ch]
21         c_code = (x + key) % ALPHABET_SIZE
22         ciphertext_chars.append(INDEX_TO_CHAR[c_code])
23
24     return "".join(ciphertext_chars)
25
26
27 def decrypt_caesar(ciphertext: str, key: int) -> str:
28     """Decrypts ciphertext with shift key k:  $m = (y - k) \bmod 31$ ."""
29     if key < 1 or key >= ALPHABET_SIZE:
30         raise ValueError(f"Key k must be in range 1..{ALPHABET_SIZE - 1}, received {key}."
31             )
32
33     plaintext_chars: List[str] = []
34     for ch in ciphertext:
35         y = CHAR_TO_INDEX[ch]
36         m_code = (y - key) % ALPHABET_SIZE
37         plaintext_chars.append(INDEX_TO_CHAR[m_code])
38
39     return "".join(plaintext_chars)
```

```

39
40 def build_permuted_alphabet(keyword: str) -> List[str]:
41     """Constructs the permuted alphabet from keyword k2 followed by remaining letters."""
42     seen = set()
43     permuted: List[str] = []
44
45     for ch in keyword:
46         if ch not in seen and ch in CHAR_TO_INDEX:
47             seen.add(ch)
48             permuted.append(ch)
49
50     for ch in ROMANIAN_ALPHABET:
51         if ch not in seen:
52             seen.add(ch)
53             permuted.append(ch)
54
55     return permuted
56
57
58 def encrypt_caesar_permuted(
59     plaintext: str, key1: int, keyword: str
60 ) -> Tuple[str, List[str]]:
61     """Encrypts plaintext using shift key k1 and permuted alphabet from keyword k2."""
62     if key1 < 1 or key1 >= ALPHABET_SIZE:
63         raise ValueError(f"Key k1 must be in range 1..{ALPHABET_SIZE - 1}, received {key1}.".")
64
65     permuted_alphabet = build_permuted_alphabet(keyword)
66     perm_char_to_index: Dict[str, int] = {ch: idx for idx, ch in enumerate(
67         permuted_alphabet)}
68
69     ciphertext_chars: List[str] = []
70     for ch in plaintext:
71         x = perm_char_to_index[ch]
72         c_code = (x + key1) % ALPHABET_SIZE
73         ciphertext_chars.append(permuted_alphabet[c_code])
74
75     return "".join(ciphertext_chars), permuted_alphabet
76
77 def decrypt_caesar_permuted(
78     ciphertext: str, key1: int, keyword: str
79 ) -> Tuple[str, List[str]]:
80     """Decrypts ciphertext using shift key k1 and permuted alphabet from keyword k2."""

```

```

81     if key1 < 1 or key1 >= ALPHABET_SIZE:
82         raise ValueError(f"Key k1 must be in range 1..{ALPHABET_SIZE - 1}, received {key1}.".")
83
84     permuted_alphabet = build_permuted_alphabet(keyword)
85     perm_char_to_index: Dict[str, int] = {ch: idx for idx, ch in enumerate(
86         permuted_alphabet)}
87
88     plaintext_chars: List[str] = []
89     for ch in ciphertext:
90         y = perm_char_to_index[ch]
91         m_code = (y - key1) % ALPHABET_SIZE
92         plaintext_chars.append(permuted_alphabet[m_code])
93
94     return "".join(plaintext_chars), permuted_alphabet

```

A.3 Command-line interface and execution logic (src/cli.py)

```

1  #!/usr/bin/env python3
2  """Command-line interface for the Romanian Caesar cipher."""
3
4  import sys
5  import os
6  import argparse
7  from typing import List
8
9  # Ensure package root is in sys.path when executed directly
10 if __package__ is None or __package__ == "":
11     sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
12     from src.alphabet import (
13         ROMANIAN_ALPHABET,
14         ALPHABET_SIZE,
15         validate_key,
16         validate_and_clean_text,
17         validate_keyword,
18     )
19     from src.caesar import (
20         encrypt_caesar,
21         decrypt_caesar,
22         build_permuted_alphabet,
23         encrypt_caesar_permuted,

```

```

24         decrypt_caesar_permuted,
25     )
26 else:
27     from .alphabet import (
28         ROMANIAN_ALPHABET,
29         ALPHABET_SIZE,
30         validate_key,
31         validate_and_clean_text,
32         validate_keyword,
33     )
34     from .caesar import (
35         encrypt_caesar,
36         decrypt_caesar,
37         build_permuted_alphabet,
38         encrypt_caesar_permuted,
39         decrypt_caesar_permuted,
40     )
41
42
43 def display_alphabet(alphabet: List[str] | tuple[str, ...]) -> None:
44     """Prints the alphabet with its numeric indices."""
45     cols = 8
46     for i in range(0, len(alphabet), cols):
47         chunk = alphabet[i : i + cols]
48         row = " ".join(f"[{i + j:2d}] {ch}" for j, ch in enumerate(chunk))
49         print(f" {row}")
50     print()
51
52
53 def prompt_key(prompt_text: str = "Enter shift key (1-30): ") -> int:
54     """Prompt loop for shift key validation."""
55     while True:
56         try:
57             raw = input(prompt_text).strip()
58         except (KeyboardInterrupt, EOFError):
59             print("\nExiting.")
60             sys.exit(0)
61
62         is_valid, key, error = validate_key(raw)
63         if is_valid:
64             return key
65         print(f"Error: {error}\n")
66
67

```

```

68 def prompt_keyword(prompt_text: str = "Enter keyword (at least 7 letters): ") -> str:
69     """Prompt loop for keyword validation."""
70     while True:
71         try:
72             raw = input(prompt_text).strip()
73         except (KeyboardInterrupt, EOFError):
74             print("\nExiting.")
75             sys.exit(0)
76
77         is_valid, keyword, error = validate_keyword(raw)
78         if is_valid:
79             return keyword
80         print(f"Error: {error}\n")
81
82
83 def prompt_text(prompt_text: str = "Enter text: ") -> str:
84     """Prompt loop for plaintext or ciphertext validation."""
85     while True:
86         try:
87             raw = input(prompt_text).strip()
88         except (KeyboardInterrupt, EOFError):
89             print("\nExiting.")
90             sys.exit(0)
91
92         is_valid, cleaned, error = validate_and_clean_text(raw)
93         if is_valid:
94             return cleaned
95         print(f"Error: {error}\n")
96
97
98 def prompt_operation() -> str:
99     """Prompt loop for choosing operation."""
100     while True:
101         try:
102             choice = input("Operation (e for encrypt, d for decrypt): ").strip().lower()
103         except (KeyboardInterrupt, EOFError):
104             print("\nExiting.")
105             sys.exit(0)
106
107         if choice in ("e", "encrypt", "1"):
108             return "encrypt"
109         elif choice in ("d", "decrypt", "2"):
110             return "decrypt"
111         print("Invalid choice. Enter 'e' to encrypt or 'd' to decrypt.\n")

```

```

112
113
114 def run_task_1_1() -> None:
115     """Interactive handler for Task 1.1."""
116     print("\nCaesar cipher (single key)")
117     op = prompt_operation()
118     key = prompt_key()
119
120     if op == "encrypt":
121         text = prompt_text("Enter plaintext: ")
122         result = encrypt_caesar(text, key)
123         print(f"\nPlaintext: {text}")
124         print(f"Ciphertext: {result}\n")
125     else:
126         text = prompt_text("Enter ciphertext: ")
127         result = decrypt_caesar(text, key)
128         print(f"\nCiphertext: {text}")
129         print(f"Plaintext: {result}\n")
130
131
132 def run_task_1_2() -> None:
133     """Interactive handler for Task 1.2."""
134     print("\nCaesar cipher with permutation (two keys)")
135     op = prompt_operation()
136     k1 = prompt_key("Enter shift key k1 (1-30): ")
137     k2 = prompt_keyword("Enter keyword k2 (at least 7 letters): ")
138
139     permuted = build_permuted_alphabet(k2)
140     print(f"\nPermuted alphabet: {' '.join(permuted)}")
141     display_alphabet(permuted)
142
143     if op == "encrypt":
144         text = prompt_text("Enter plaintext: ")
145         result, _ = encrypt_caesar_permuted(text, k1, k2)
146         print(f"Plaintext: {text}")
147         print(f"Ciphertext: {result}\n")
148     else:
149         text = prompt_text("Enter ciphertext: ")
150         result, _ = decrypt_caesar_permuted(text, k1, k2)
151         print(f"Ciphertext: {text}")
152         print(f"Plaintext: {result}\n")
153
154
155 def run_menu() -> None:

```

```

156     """Main interactive menu loop."""
157     print("Caesar cipher (Romanian alphabet)")
158     while True:
159         print("1. Caesar cipher (single key)")
160         print("2. Caesar cipher with permutation (two keys)")
161         print("3. Display Romanian alphabet")
162         print("0. Exit")
163
164         try:
165             choice = input("\nSelect an option: ").strip()
166         except (KeyboardInterrupt, EOFError):
167             print("\nExiting.")
168             break
169
170         if choice == "1":
171             run_task_1_1()
172         elif choice == "2":
173             run_task_1_2()
174         elif choice == "3":
175             print(f"\nRomanian alphabet (length {len(ROMANIAN_ALPHABET)}):")
176             display_alphabet(ROMANIAN_ALPHABET)
177         elif choice in ("0", "q", "exit"):
178             break
179         else:
180             print("Invalid option. Please enter 0, 1, 2, or 3.\n")
181
182
183     def main() -> None:
184         parser = argparse.ArgumentParser(
185             description="Romanian Caesar cipher implementation",
186         )
187         parser.add_argument(
188             "--task",
189             choices=["1.1", "1.2"],
190             help="task to execute (1.1 for single key, 1.2 for two keys)",
191         )
192         parser.add_argument(
193             "--mode",
194             choices=["encrypt", "decrypt"],
195             help="operation mode",
196         )
197         parser.add_argument(
198             "--key", "-k",
199             type=int,

```

```

200         help="shift key (1-30)",
201     )
202     parser.add_argument(
203         "--keyword", "-w",
204         type=str,
205         help="permutation keyword (at least 7 Romanian letters)",
206     )
207     parser.add_argument(
208         "--text", "-t",
209         type=str,
210         help="input text",
211     )
212     parser.add_argument(
213         "--show-alphabet",
214         action="store_true",
215         help="display the Romanian alphabet and exit",
216     )
217     parser.add_argument(
218         "--interactive", "-i",
219         action="store_true",
220         help="launch interactive menu",
221     )
222
223     args = parser.parse_args()
224
225     if len(sys.argv) == 1 or args.interactive:
226         run_menu()
227         return
228
229     if args.show_alphabet:
230         print(f"Romanian alphabet (length {len(ROMANIAN_ALPHABET)}):")
231         display_alphabet(ROMANIAN_ALPHABET)
232         return
233
234     if not args.task or not args.mode or args.key is None or not args.text:
235         parser.error("Non-interactive mode requires --task, --mode, --key, and --text.")
236
237     ok_k, valid_k, err_k = validate_key(args.key)
238     if not ok_k:
239         print(f"Error: {err_k}", file=sys.stderr)
240         sys.exit(1)
241
242     ok_t, clean_t, err_t = validate_and_clean_text(args.text)
243     if not ok_t:

```

```

244     print(f"Error: {err_t}", file=sys.stderr)
245     sys.exit(1)
246
247 if args.task == "1.1":
248     if args.mode == "encrypt":
249         print(encrypt_caesar(clean_t, valid_k))
250     else:
251         print(decrypt_caesar(clean_t, valid_k))
252 elif args.task == "1.2":
253     if not args.keyword:
254         parser.error("Task 1.2 requires --keyword.")
255     ok_w, valid_w, err_w = validate_keyword(args.keyword)
256     if not ok_w:
257         print(f"Error: {err_w}", file=sys.stderr)
258         sys.exit(1)
259
260     perm = build_permuted_alphabet(valid_w)
261     print(f"Permuted alphabet: {''.join(perm)}")
262     if args.mode == "encrypt":
263         res, _ = encrypt_caesar_permuted(clean_t, valid_k, valid_w)
264         print(f"Ciphertext: {res}")
265     else:
266         res, _ = decrypt_caesar_permuted(clean_t, valid_k, valid_w)
267         print(f"Plaintext: {res}")
268
269
270 if __name__ == "__main__":
271     main()

```